

AY 2025/2026 S1

EE6427 Video Signal Processing

Part 6 Emerging Topics in Video Signal Processing

Dr Yap Kim Hui

Room: S2-B2b-53

Tel: 6790 4339

Email: ekhyap@ntu.edu.sg



References

- Rishi Bommasani et. al. On the Opportunities and Risks of Foundation Models, arXiv:2108.07258.
- CMU Course11-785 Introduction to Deep Learning.
- OpenAI, <https://openai.com/>
- Pranab Sahoo et. al. A Systematic Survey of Prompt Engineering in Large Language Models: Techniques and Applications, arXiv:2402.07927.
- Stanford Lecture Notes, CS231n: Convolutional Neural Networks for Visual Recognition.

Part 6 Outline

- The part will cover the following topics:
 - Section I: Foundation Models (FMs)
 - Section II: Large Language Models (LLMs)

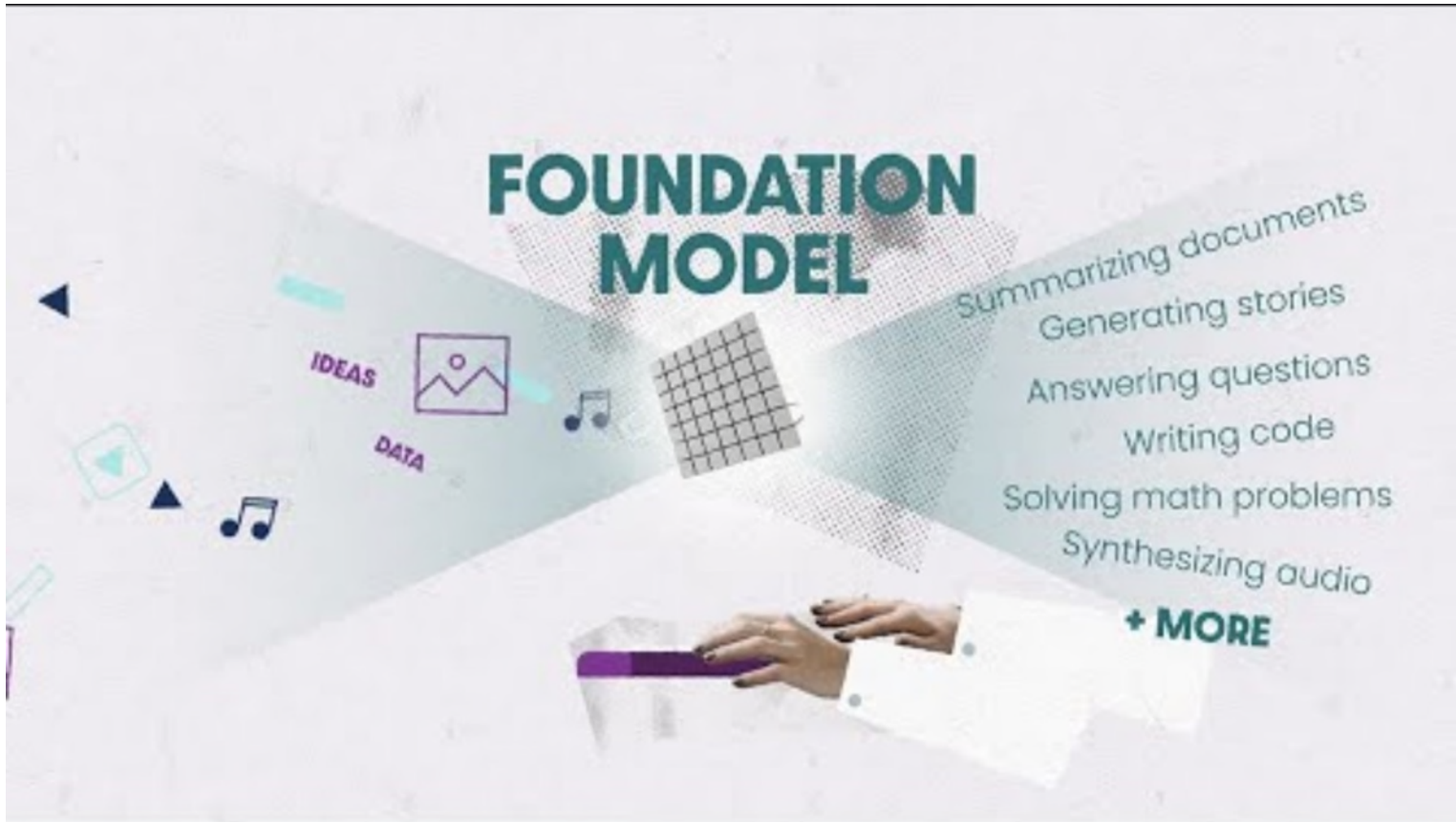
Section I

Foundation Models (FMs)

Section I Overview

- The section will cover the following topics:
 - Basics of Foundation Models (FMs)
 - Adapting / Finetuning FMs

Foundation Models



Basics of Foundation Models (FMs)

What is Foundation Model (FM)?

- A new paradigm of AI based on large-scale pre-training and downstream adaptation.
- Pretraining:
 - A model is trained on a large-scale diverse data.
 - Typically use self-supervised learning to alleviate expensive data collection and annotation.
- Adaptation / fine-tuning:
 - Pre-trained models are adapted for subsequent downstream tasks.
- Examples:
 - Large Language Models (e.g., GPT), Vision-Language Models (e.g., CLIP), Multimodal LLM (GPT-4o, LLaVA).

Key Ideas of FMs

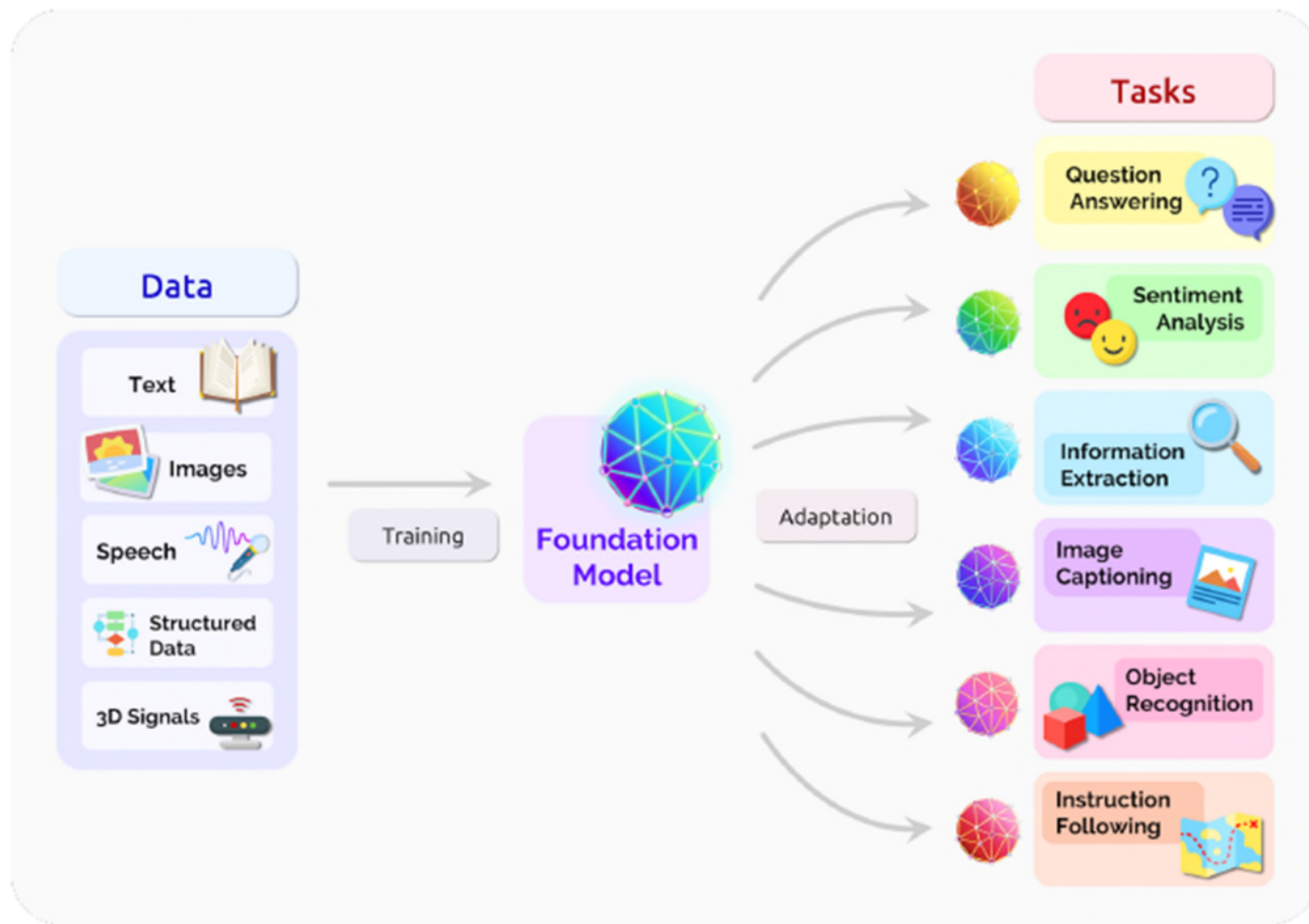


Fig. 2. A foundation model can centralize the information from all the data from various modalities. This one model can then be adapted to a wide range of downstream tasks.

Different Stages of FMs

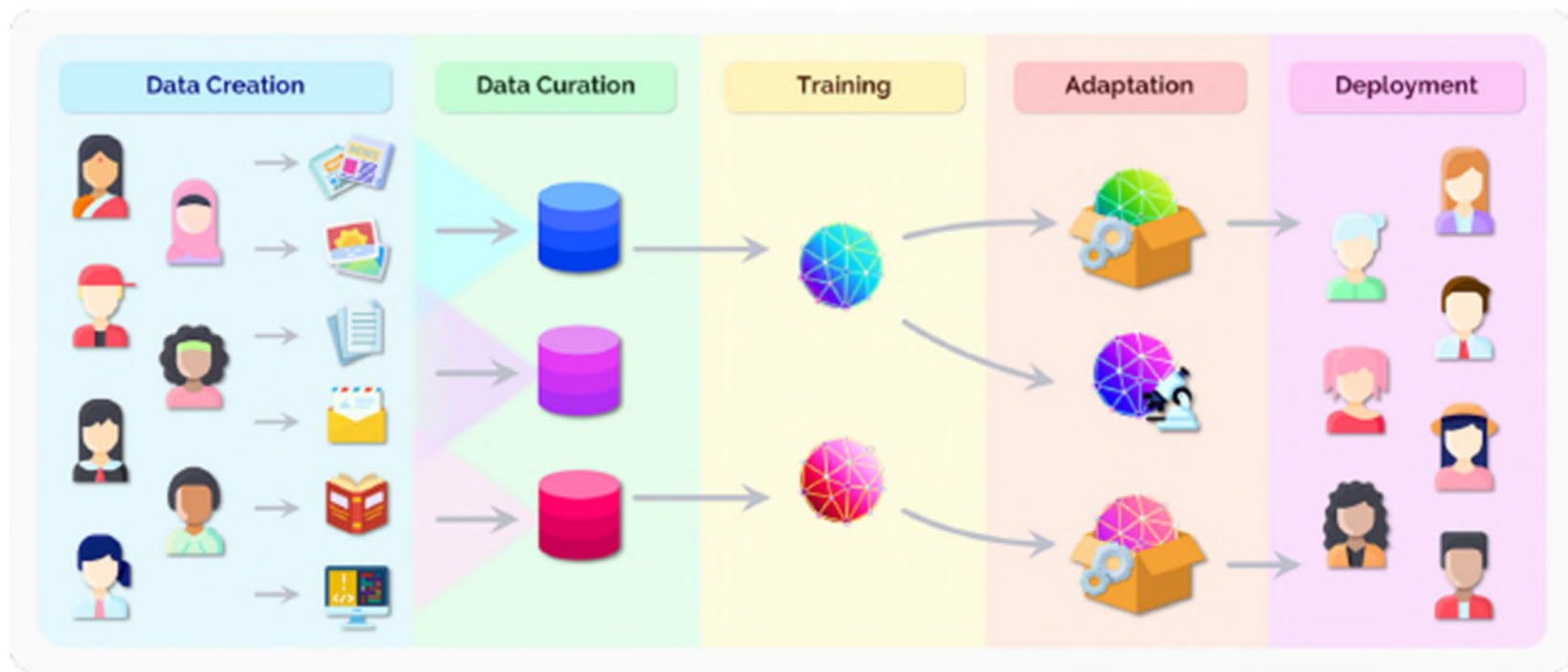
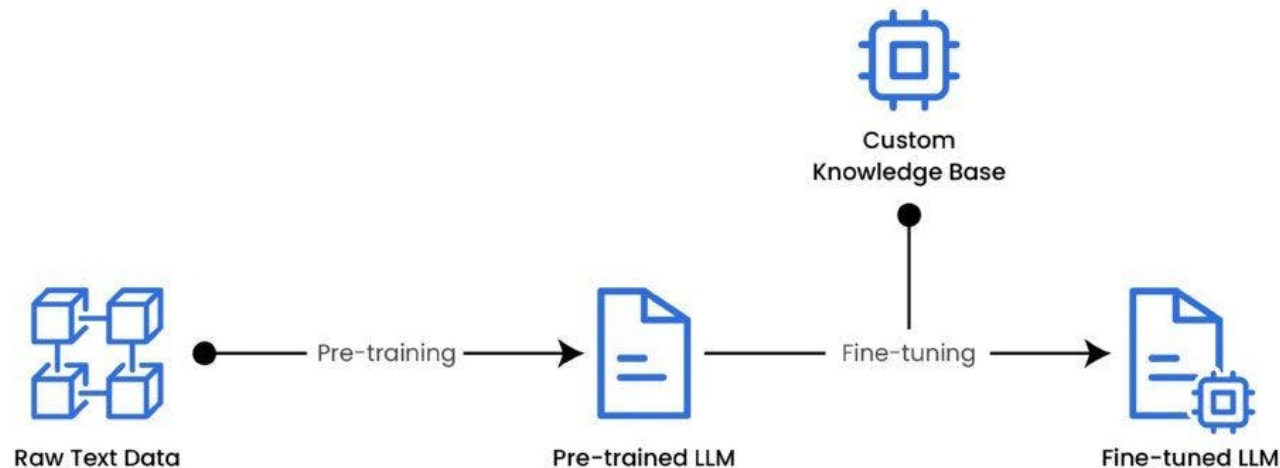


Fig. 3. Before reasoning about the social impact of foundation models, it is important to understand that they are part of a broader ecosystem that stretches from data creation to deployment. At both ends, we highlight the role of people as the ultimate source of data into training of a foundation model, but also as the downstream recipients of any benefits and harms. Thoughtful data curation and adaptation should be part of the responsible development of any AI system. Finally, note that the deployment of adapted foundation models is a decision separate from their construction, which could be for research.

Adapting / Finetuning FMs

What is Fine-tuning?

- A process of adjusting parameters of a pre-trained model for a specific downstream task / use case.
- Pre-trained models possess vast & diverse knowledge but lack specialization in specific areas.
- Fine-tuning addresses this gap by adapting the model to learn from domain-specific data to make it more accurate and effective for targeted applications.



Fine-tuning Process

- Data preparation
 - Data cleaning, handling missing values, formatting text to match model's input requirements, data augmentation, etc.
- Pre-trained model selection
 - Understand architecture, input/output specifications of models.
 - Consider factors such as model size, training data, performance, etc.
- Method selection for fine-tuning
 - Choose suitable fine-tuning or adaptation method for specific application.
- Validation
 - Evaluate a fine-tuned model's performance on target validation set.
- Model Iteration
 - Repeat the process above, if necessary.
- Deployment
 - Position the fine-tuned model in real-world target application.

Adaptor Tuning

- A technique to fine-tune large pre-trained models.
- Introduce adaptors which are small network layers.
- Train only the adaptors.
- Useful for limited computational resources or labelled data.

Key Steps in Adaptor Tuning

- Introduce small lightweight adaptors (networks) between layers of pre-trained model.
- During fine-tuning, only the adaptors are trained while the original weights (parameters) of the foundation model are frozen.
- Significantly reduce the number of parameters that need to be updated, making fine-tuning more efficient in terms of computation and memory usage.

Adaptor Tuning

- A sample adapter consisting of 2 FC layers & nonlinear activation added to Transformer.

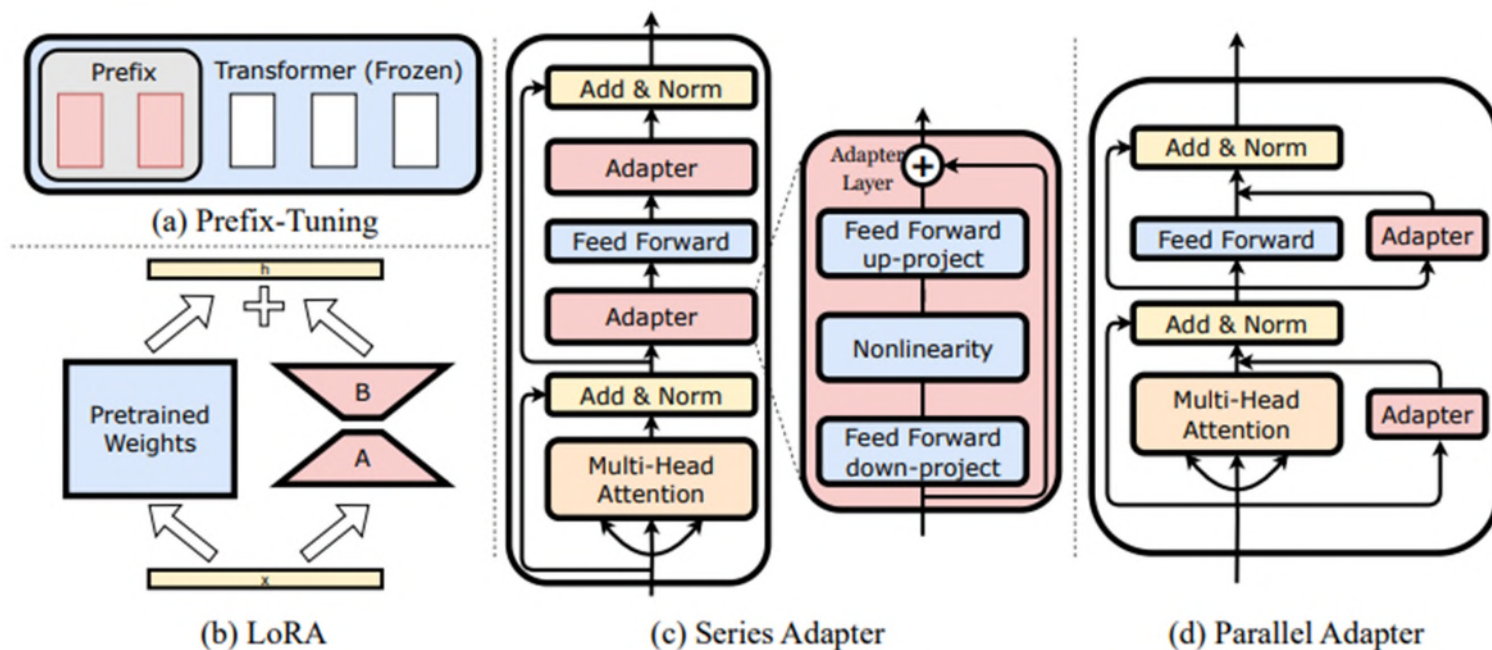


Figure 1: A detailed illustration of the model architectures of three different adapters: (a) Prefix-Tuning, (b) LoRA, (c) Series Adapter, and (d) Parallel Adapter.

What is Low-Rank Adaptation (LoRA)?

- An efficient fine-tuning method for foundation models such as large language models.
- Low computational overhead and parameter update.
- E.g., GPT-3: 175B fine-tuned with Adam vs reduction of trainable parameters by 10,000X and GPU memory requirement by 3X with LoRA.

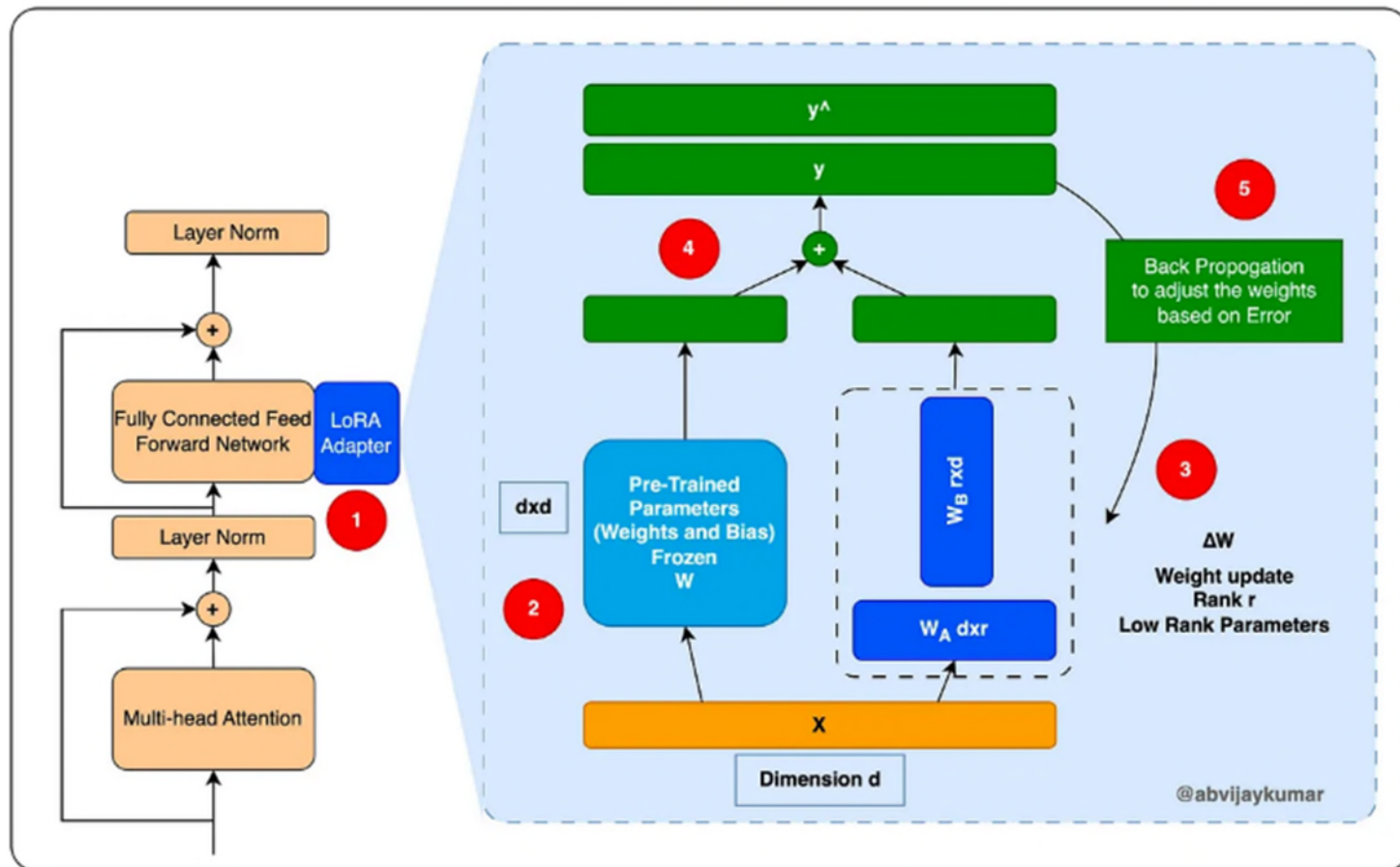
Steps of LoRA

- Select a pre-trained FM.
- Identify target layers for adaptation.
 - Common selected layer: attention layer in a transformer.
- Introduce low-rank matrices for low-rank decomposition.
 - For each selected layer, introduce two low-rank matrices A and B that approximate changes to the original weight matrix W .

$$W' = W + A \times B$$

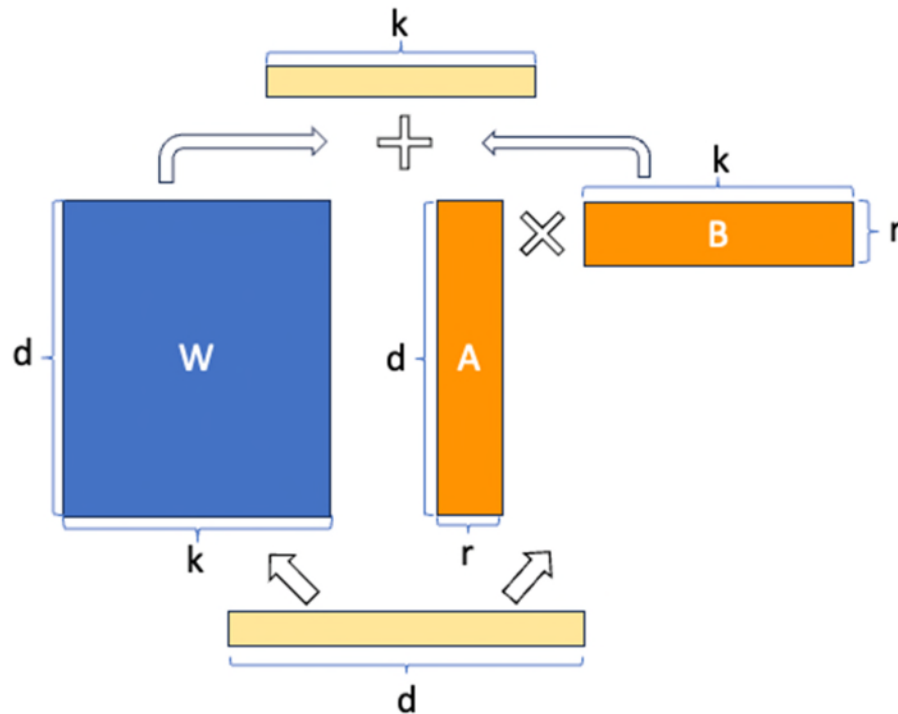
- Fine-tune the model.
 - Freeze the original weight matrix W . Update only the low-rank matrices A and B using target use case data.
- Evaluate the adapted model performance.
- Optimization
 - Hyperparameter tuning, model evaluation.
- Deploy and Monitor

Steps of LoRA



A neural network utilizing a LoRA Adapter to update pre-trained network parameters efficiently [Source: @abvijaykumar]

Visualization of LoRA



Section I Overview

- The section covers the following topics:
 - Basics of Foundation Models (FMs)
 - Adapting / Finetuning FMs

Section II

Large Language Models (LLMs)

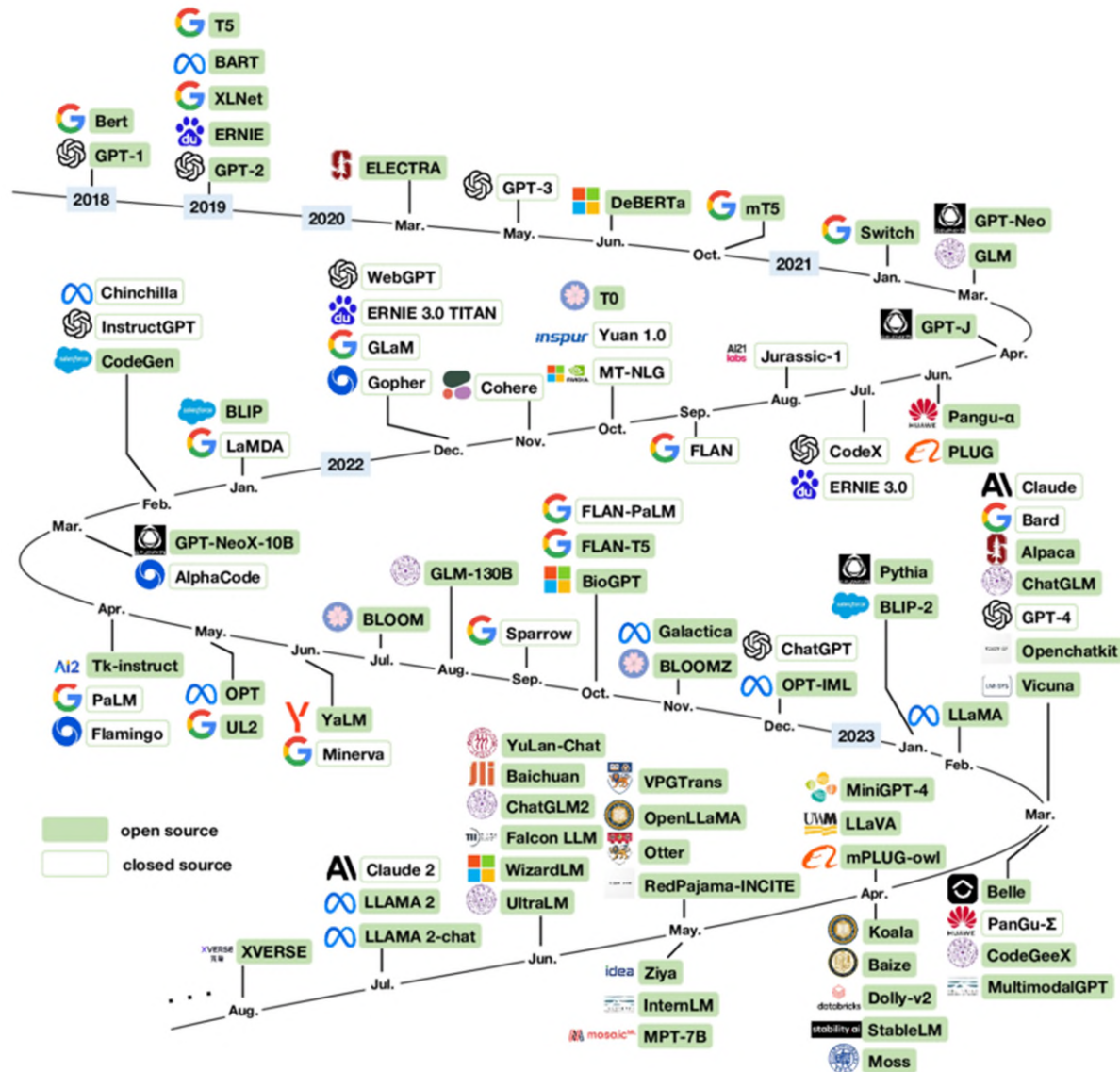
Section II Overview

- The section covers the following topics:
 - Introduction
 - BERT
 - GPT

What are LLMs?

- AI models designed to understand, generate, and manipulate human language.
- Key Features:
 - Massive Scale: billions of parameters, enabling them to handle a wide range of linguistic tasks.
 - Training Data: trained on massive amounts of text data from various sources, allowing them to learn different aspects of human language.
 - Capabilities: can handle different tasks such as language translation, question answering, text generation, sentiment analysis, etc.
 - Applications: natural language processing, chatbots, content creation, language understanding, etc.
 - Challenges: bias, misuse, societal impacts, etc.

Timelines of Popular LLMs

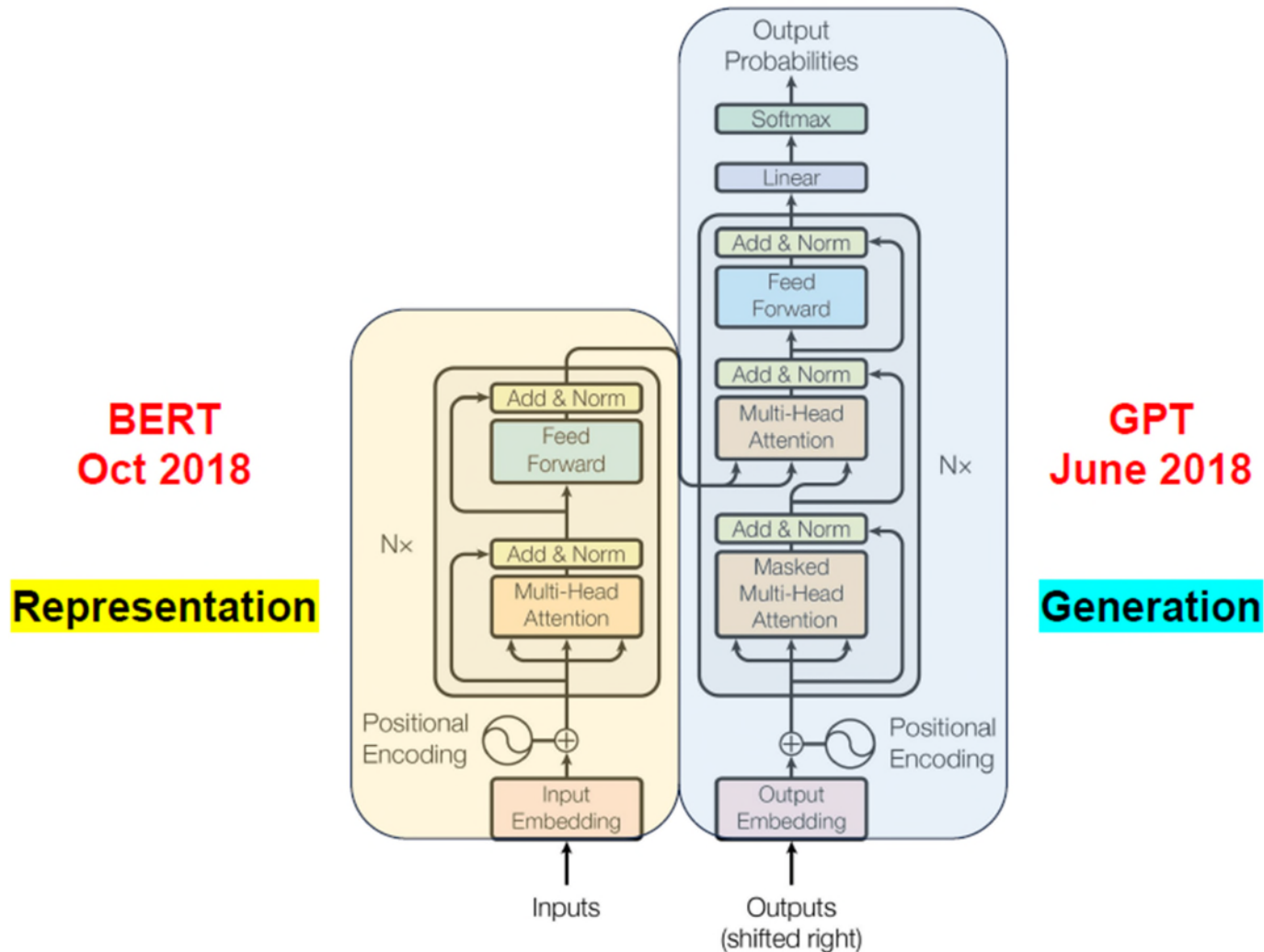


Bidirectional Encoder Representations from Transformers (BERT)

Introduction

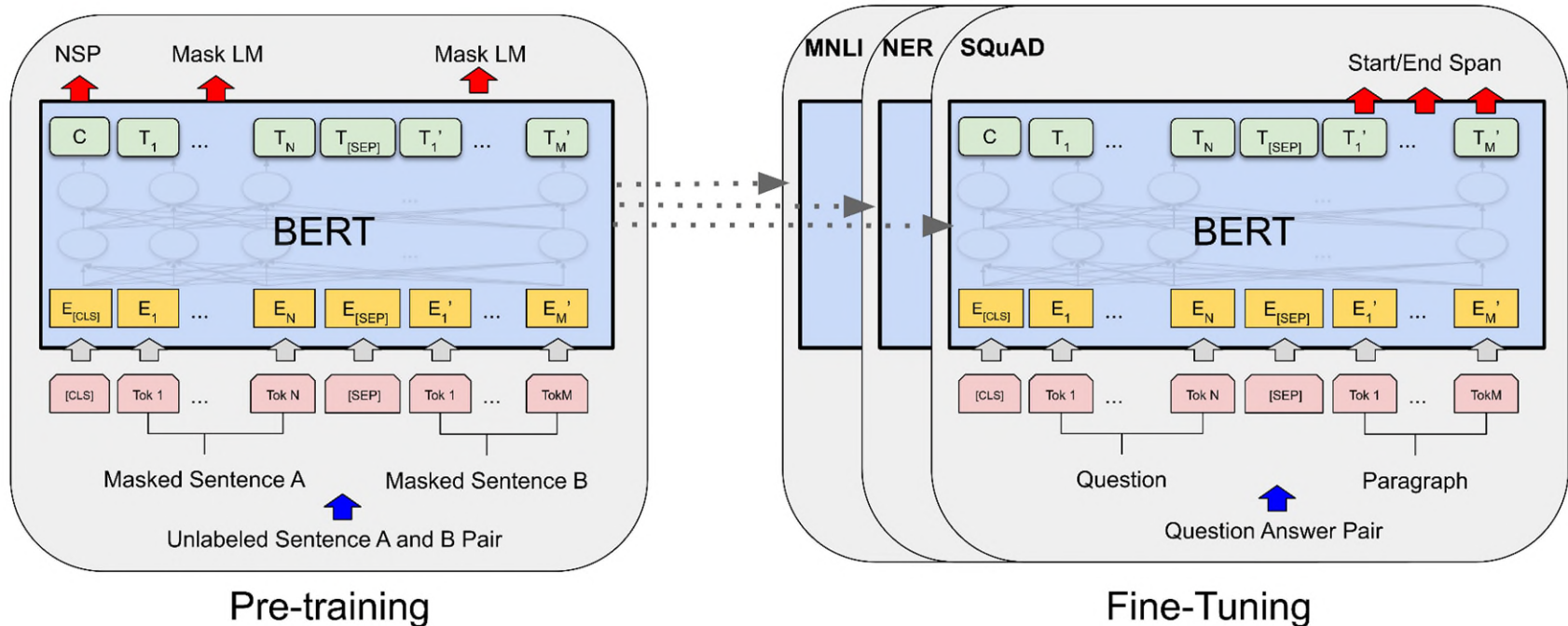
- An NLP model developed by Google around 2018.
- A transformer encoder with bidirectional self-attention.
- Understand bidirectional context relationship between words.
- Can generate contextualized word embeddings.
- Two versions with varying model sizes and complexities:
 - BERT-base (110M parameters)
 - BERT-large (340M parameters)
- Variants: RoBERTa, ALBERT, DistilBERT, and ELECTRA.

Transformer: BERT + GPT



BERT Training

- Two stages:
 - Stage 1: pretrain BERT to understand language.
 - Stage 2: fine-tune BERT to learn specific downstream task.



Stage 1: Pre-Training

- Use unsupervised / self-supervised learning.
- Goal: to learn language structure & context.
- 2 key training tasks in pre-training:
 - Masked Language Model (MLM)
 - Predict masked words from surrounding words.
 - Aim to achieve context learning.
 - Next Sentence Prediction (NSP)
 - Predict if a sentence follows another in a text pair.
 - Learn logical context between sentences.
- BERT pretraining corpus
 - English Wikipedia - 2,500 million words.
 - Book Corpus - 800 million words.

BERT Input Representation

- Token Embedding: WordPieces, 30K vocabulary.
- Segment embedding: sentence numbering.
- Position embedding: capture position of words in a sentence.

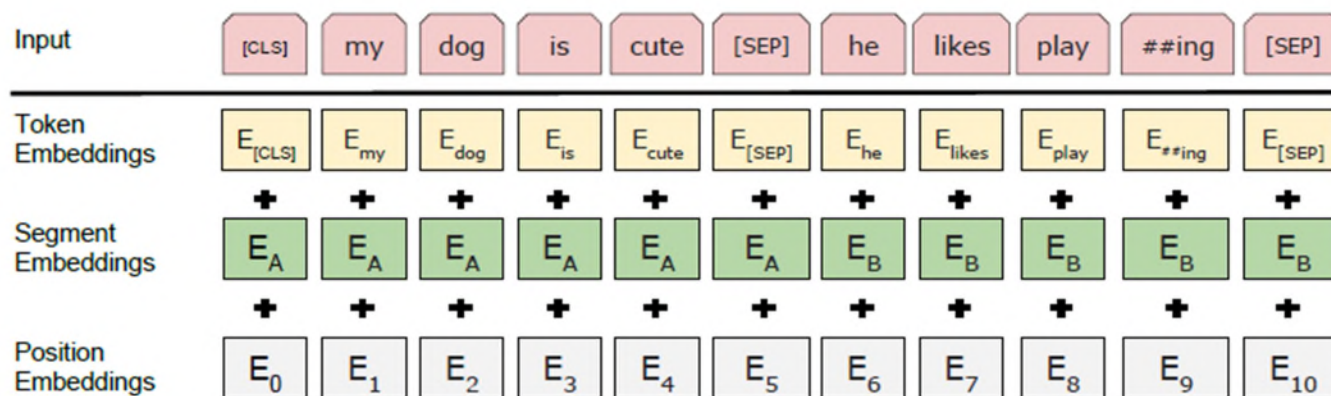
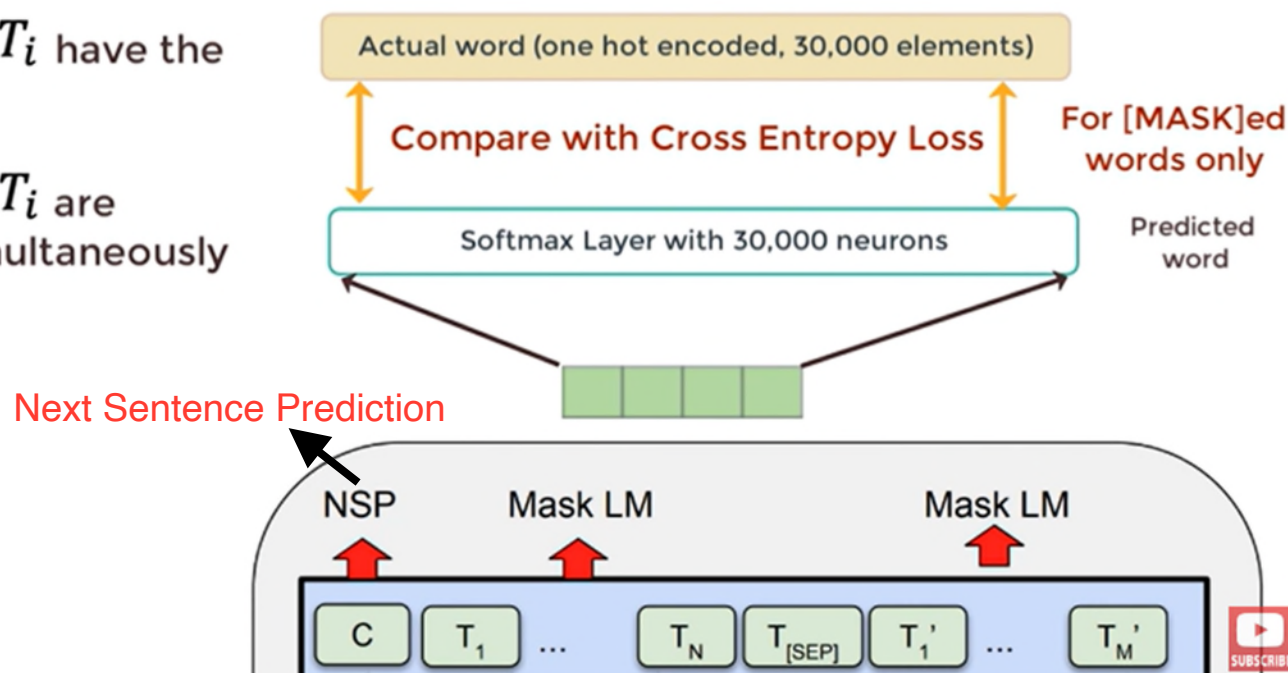


Figure 2: BERT input representation. The input embeddings are the sum of the token embeddings, the segmentation embeddings and the position embeddings.

Stage 1: Pre-Training

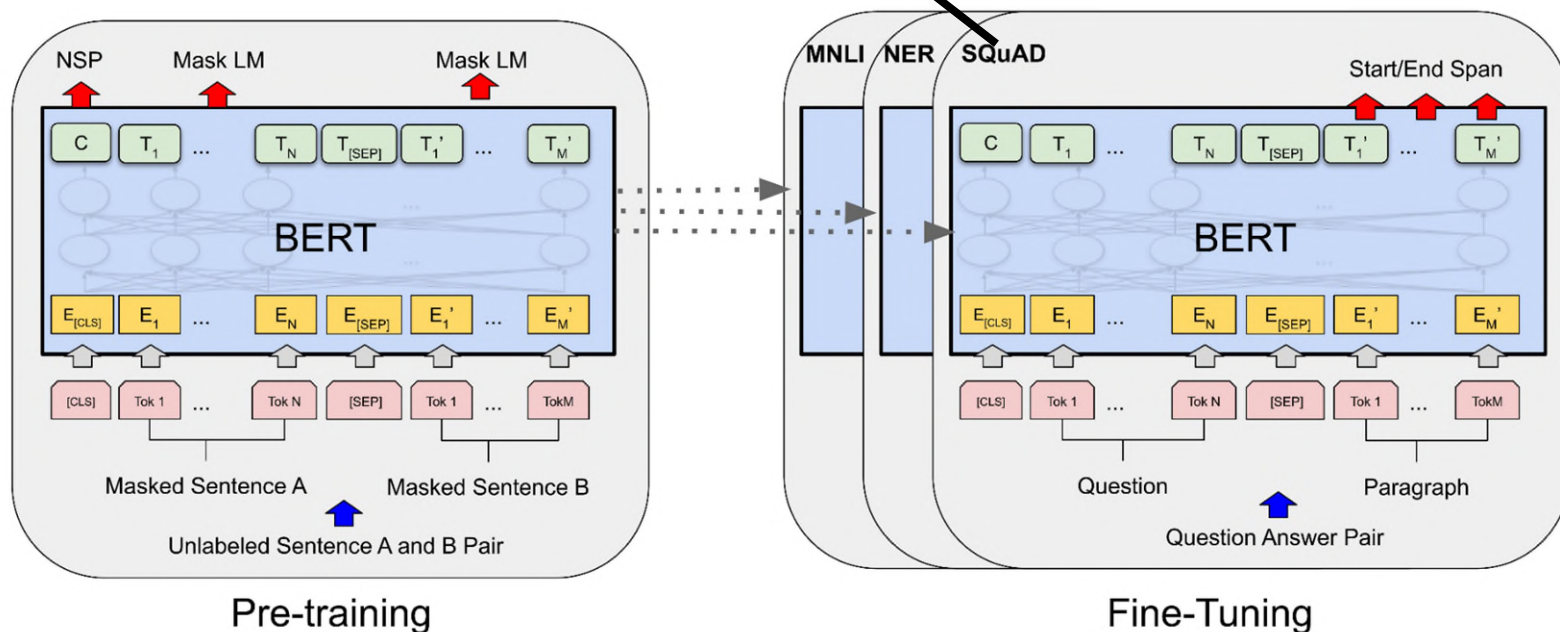
Word vectors T_i have the same size.

Word vectors T_i are generated simultaneously



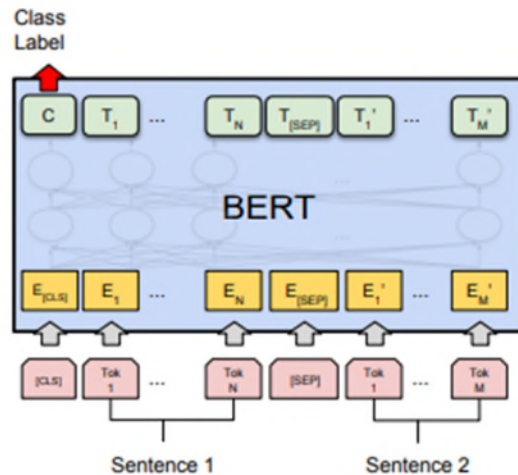
Stage 2: Finetuning for Specific Tasks

- Use supervised learning: add a task-specific module after the last encoder layer so that it can map to the target task.
- Example: for **Question Answering Task**, train two extra vectors to mark the beginning and end of answer from the paragraph.
- Sample downstream tasks: sentiment analysis, text summarization, etc.

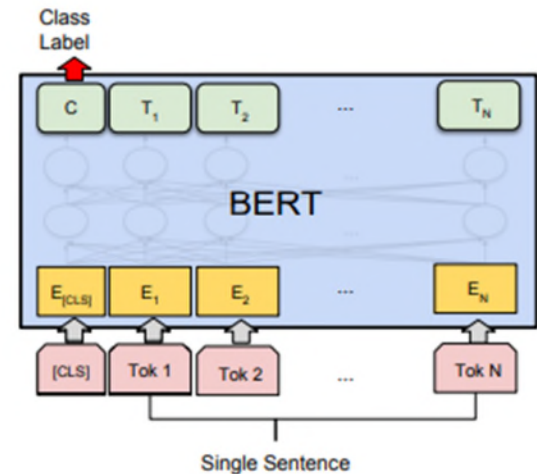


Performance Evaluation

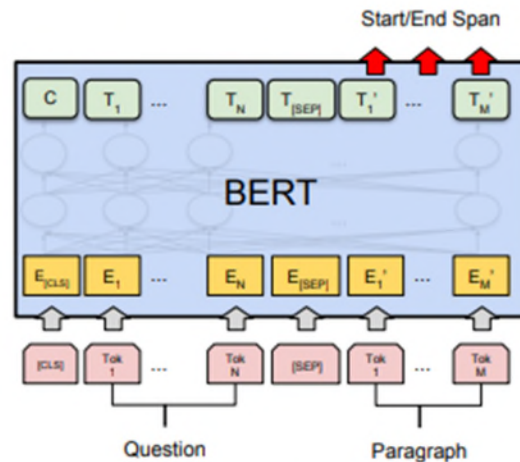
- General Language Understanding Evaluation (GLUE)
 - Sentence pair task.
 - Single sentence classification task.
- Stanford Question Answering Dataset (SQuAD)



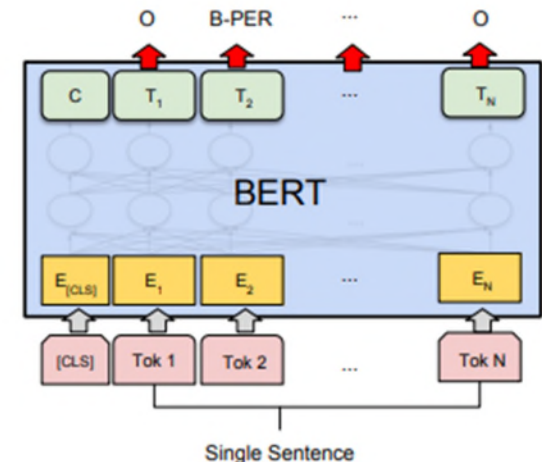
(a) Sentence Pair Classification Tasks: MNLI, QQP, QNLI, STS-B, MRPC, RTE, SWAG



(b) Single Sentence Classification Tasks: SST-2, CoLA



(c) Question Answering Tasks: SQuAD v1.1



(d) Single Sentence Tagging Tasks: CoNLL-2003 NER

Performance

- Outperform many task-specific models.

System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	Average -
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
BERT _{BASE}	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
BERT _{LARGE}	86.7/85.9	72.1	92.7	94.9	60.5	86.5	89.3	70.1	82.1

Table 1: GLUE Test results, scored by the evaluation server (<https://gluebenchmark.com/leaderboard>). The number below each task denotes the number of training examples. The “Average” column is slightly different than the official GLUE score, since we exclude the problematic WNLI set.⁸ BERT and OpenAI GPT are single-model, single task. F1 scores are reported for QQP and MRPC, Spearman correlations are reported for STS-B, and accuracy scores are reported for the other tasks. We exclude entries that use BERT as one of their components.

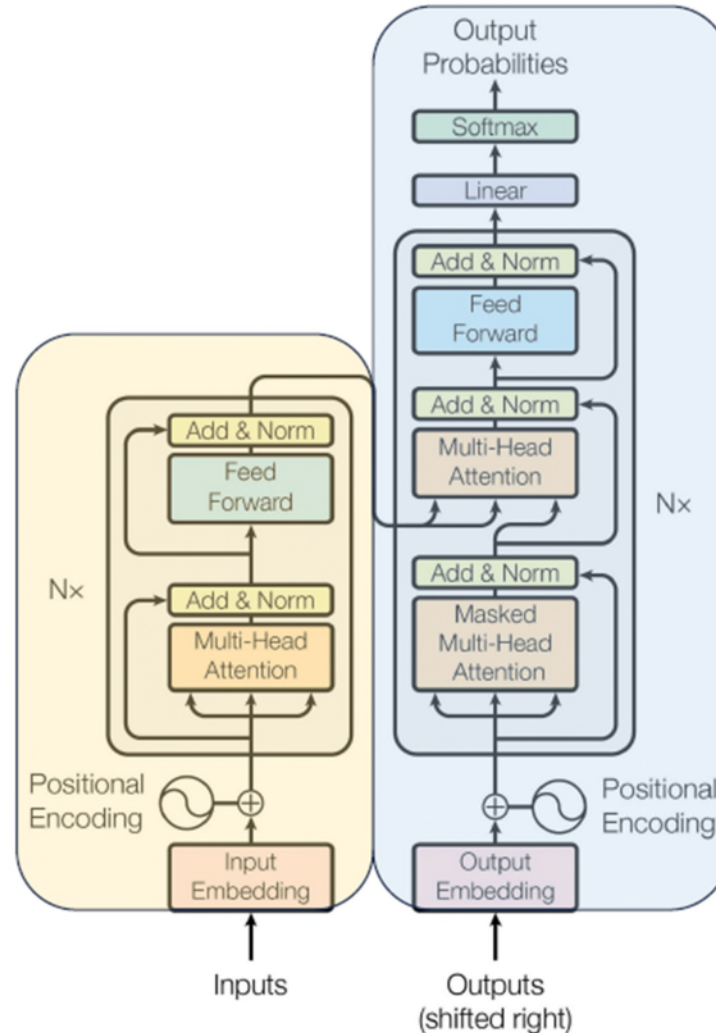
Generative Pretrained Transformer (GPT)

Transformer: BERT + GPT



BERT
Oct 2018

Representation



GPT
June 2018

Generation

What is GPT?

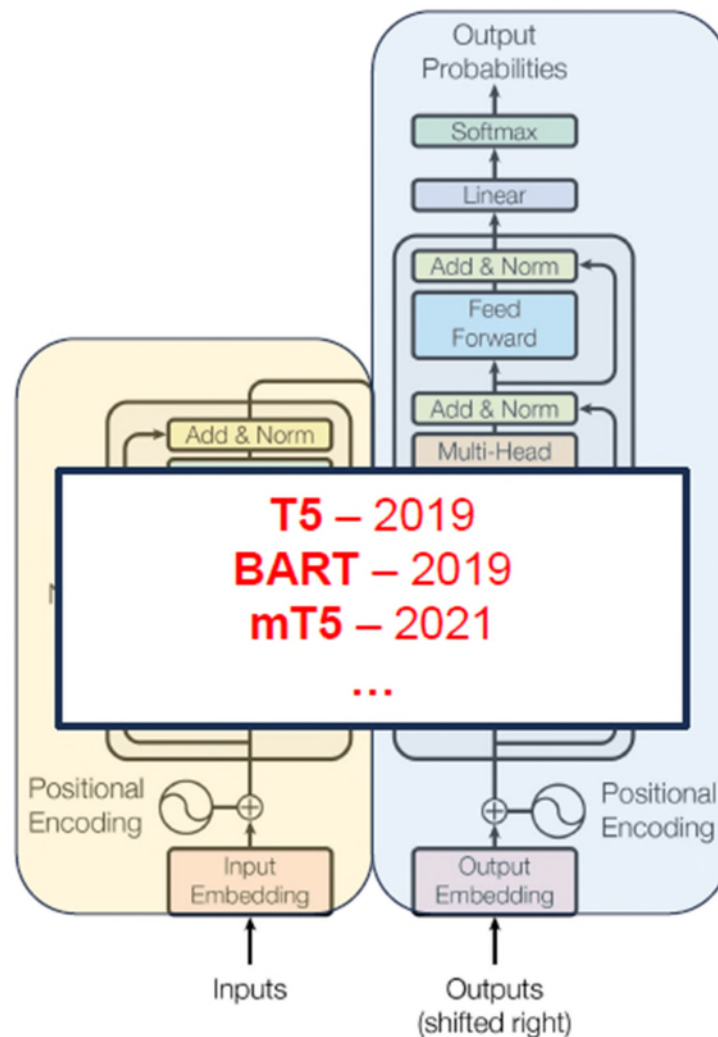
- Generative Pre-trained Transformer (GPT) is an AI model pre-trained on vast amounts of texts to learn to generate human-like text.
- It leverages self-supervised learning to use large amounts of unlabelled data to pretrain an LLM.
- Sample applications: text generation, translation, and conversation, making it highly versatile for various language applications.
- A transformer decoder.
- GPT pretraining corpus: English Wikipedia & book corpus.
- GPT pretraining: predict the next token from the previous tokens.
- GPT fine-tuning: task-specific prompt text for the model to learn.

Recent LLM Development

BERT – 2018
DistilBERT – 2019
RoBERTa – 2019
ALBERT – 2019
ELECTRA – 2020
DeBERTa – 2020

...

Representation



GPT – 2018
GPT-2 – 2019
GPT-3 – 2020
GPT-Neo – 2021
GPT-3.5 (ChatGPT) – 2022
LLaMA – 2023
GPT-4 – 2023

...

Generation

LLM Architectures

- Encoder-only (e.g., BERT)
 - Pre-training: Masked Language Modelling (MLM), Next Sentence Prediction (NSP).
 - Good for classification tasks.
- Decoder-only (e.g., GPT)
 - Pre-training: Next word / token prediction.
 - Good for generation tasks.
 - Better generalization after pre-training.
- Encoder-decoder (e.g., Text-to-Text Transfer Transformer or T5)
 - Pre-training : provide text as input and train it to generate target text.
 - Good for tasks like machine translation.

ChatGPT



GPT-4o

GPT-4o

- More natural human-computer interaction.
- Accept as input any combination of text, audio, image, and video and generate any combination of text, audio, and image outputs.
- A single end-to-end model across text, vision, and audio.
- Fast: similar to human response time in a conversation.
- Improve on non-English languages.
- Better at vision and audio understanding compared to existing models.

GPT-4o Voice & Vision



GPT-4o



GPT-4o Math Problem Solving



GPT-4o



Section II Summary

- The section covers the following topics:
 - Introduction
 - BERT
 - GPT

Part 6 Summary

- The part covered the following topics:
 - Section I: Foundation Models (FMs)
 - Section II: Large Language Models (LLMs)